

面向 AI 原生系统的智能运维

第 11 讲 | AIOps 平台架构与工程实践

连接监控、告警、自动化、配置与故障处置

陈鹏飞 | 中山大学

从第 10 讲继续：转型最后要落在哪里？



第 10 讲 为什么转、先做什么、如何验收。

本讲 平台怎样把能力串成共同语言。

后两讲 大模型平台进阶与 RCA Agent 实战。

平台容量算例：接入峰值结束后，积压要多久排空？

假设事件到达速率在 10 分钟内为 12,000 条/秒，处理能力恒定为 10,000 条/秒，初始无积压。

阶段	到达 / 处理速率	队列变化	结果
峰值 600 秒	12,000 / 10,000	$(12000 - 10000) \times 600$	积压 1,200,000 条
峰值后	8,000 / 10,000	净排空 2,000 条/秒	需要再等 600 秒
每条 1 KiB	只计事件载荷	约 1.14 GiB	不含索引、副本及日志

$$B_{t+\Delta t} = \max(0, B_t + (\lambda - \mu)\Delta t)$$

推导与判断 峰值结束不代表数据已经追平。界面与 Agent 应看到事件新鲜度和积压量；若峰值后到达速率仍不低于处理能力，队列无法自行排空。

算例使用假设数据。

开场问题：为什么有了监控，事故仍然难以处置？

监控已经告诉你

- ▶ 某个服务的错误率上升；
- ▶ 某个节点 CPU 接近上限；
- ▶ 某条 Trace 变慢；
- ▶ 某个版本刚刚发布。

事故仍然需要回答

- ▶ 这些信号是不是同一个事件？
- ▶ 影响的是哪个业务对象？
- ▶ 先查什么、谁来审批、动作会不会扩大影响？
- ▶ 恢复后如何证明真的恢复并保存已核对的结论？

本讲主线 AIOps 平台的核心不是更多图表，而是让信号、对象、责任、动作和证据共享同一条状态链。

范围与非目标：先把平台边界画清楚

本讲覆盖

- ▶ 总体架构、数据流和能力分层；
- ▶ 观测、告警、配置、知识和自动化的接口约定；
- ▶ 蓝鲸式平台模块组合与工程治理；
- ▶ 可观测、可审计、可回放、可升级。

留给后续

- ▶ 第 12 讲：大模型驱动的平台架构进阶；
- ▶ 第 13 讲：RCA Agent 的详细实现；
- ▶ 具体模型选型、提示词和评测代码；
- ▶ 生产环境的厂商版本承诺。

架构课的目标是建立可讨论的边界，而不是一页图里塞入所有组件。

2 学时路线图

1. 平台问题：为什么独立工具会形成新的断点；
2. 总体架构：数据、对象、事件、证据与动作如何流动；
3. 能力分层：从监控到处置的最小平台接口约定；
4. 蓝鲸实践：模块化、插件化、租户、安全和升级；
5. 综合案例：接收事件、查询原因、处置故障并记录复盘结果；
6. 平台设计：给出一张可验收的平台蓝图。

平台：统一数据、协调操作、明确分工

内容顺序：先理解平台，再比较研究方法

1. 第 8–47 页：从缓存事故建立对象、事件、数据流与权限模型。
2. 第 48–80 页：先讲各能力如何配合，再用算例说明检测、关联和动作检查。
3. 第 81–110 页：蓝鲸实践与平台蓝图，选择一个案例完整推演。
4. 第 111–125 页：论文拓展；比较方法如何接入已有平台接口。
5. 第 126 页：数据库与事件总线不一致时，检查平台怎样避免丢事件。

衔接 第 12 讲在这套平台上加入模型服务、检索和 Agent；第 13 讲实现一次诊断任务。

一、平台问题

监控工具很多，但企业需要一条可协作、可验证的状态链

AIOps 平台到底是什么？



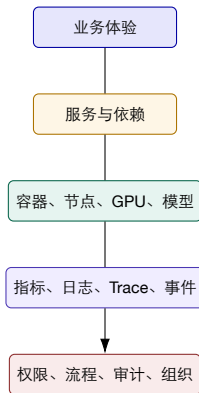
- ▶ 不是某一个算法，也不是单独的监控大盘；
- ▶ 是把观测数据、运维对象、流程、工具、知识和责任组织起来的系统；
- ▶ 它应当能把“发现异常”推进到“验证恢复并学习”。

监控大盘与 AIOps 平台的差别

维度	监控大盘	AIOps 平台
核心对象	指标、图表和告警	服务、版本、变更、事件、工单和动作
主要问题	现在是否异常？	影响谁、为什么、下一步做什么？
跨系统能力	依赖人工切换	统一身份、时间和查询入口
执行能力	通常只读	建议、审批、受控执行、回滚和验证
持续改进	仪表盘维护	规则、知识、工具和评测版本化

判断标准 如果平台不能把一次事件变成可追踪的状态机，它大概率仍然只是工具集合。

企业级平台的五层上下文



故障处置必须跨层：业务影响决定优先级，运行时证据解释原因，控制面约束动作。

平台的四类企业约束

规模 多集群、多租户、多地域，数据量和调用量持续增长。

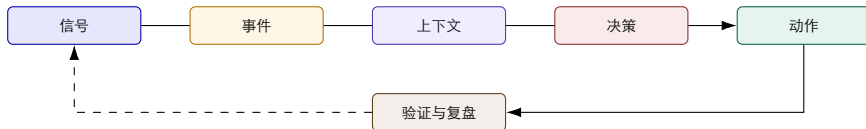
异构 云、IDC、容器、GPU、模型和传统系统并存。

责任 平台、应用、网络、安全和业务共同承担结果。

合规 最小权限、审计留痕、数据隔离和可回滚。

工程含义 “能跑通”只是功能验收；企业还要求可扩展、可解释、可审计、可恢复和可升级。

平台如何接收事件、执行处置并检查结果？



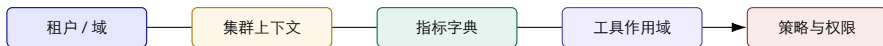
- ▶ 信号是原始观测，事件是可管理的状态；
- ▶ 上下文把对象、依赖、变更、知识和责任补齐；
- ▶ 动作不是命令字符串，而是有权限、审批、超时、回滚和结果的工作单元。

同一个事件，四类角色看见什么？

角色	关心的问题	平台应提供的视图
业务负责人 应用 SRE 平台工程 治理/安全	用户影响和恢复承诺是否满足？ 哪个版本、依赖或资源触发了异常？ 系统是否按容量和策略运行？ 谁查了什么、执行了什么？	业务对象、SLO、影响范围、沟通状态 调用链、变更、指标、日志、拓扑 集群、节点、队列、成本、自动化结果 身份、权限、审批、审计、回滚证据

同一事件可以有多个视图，但不能有多个互相矛盾的事件身份。

多租户与多集群：上下文隔离不是附加功能

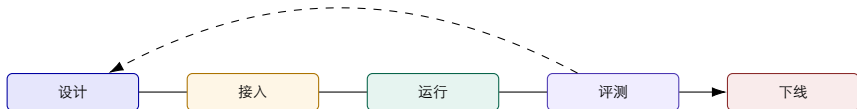


- ▶ ‘social-network / general’ 和 ‘vllm-t4 / llm’ 不应共享同一套默认上下文；
- ▶ 租户、集群和业务域共同决定查询范围、指标语义、Skill 和执行端点；
- ▶ 隔离失败既是数据泄露风险，也是错误动作风险。

六条平台设计原则

1. 对象优先：先定义服务、版本、集群、事件和工单，再堆图表；
2. 先约定接口：每个数据源、工具和动作都有输入、输出、错误和权限语义；
3. 证据优先：结论必须能回到时间范围、查询和原始信号；
4. 状态优先：消息只是通知，事件状态机才是协作依据；
5. 安全默认：查询、建议、执行分级，危险动作默认需接管；
6. 可演进：模块可替换，资产可版本化，失败可回放。

平台也有生命周期



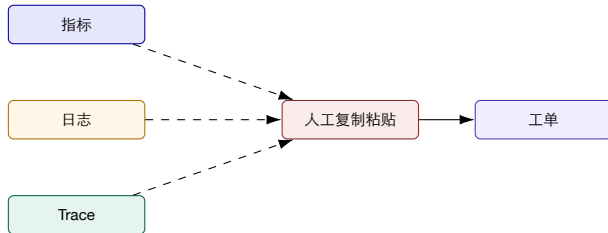
- ▶ 新服务接入时需要对象、指标、日志、Trace、负责人和 Runbook;
- ▶ 运行中需要数据质量、告警质量、自动化成功率和成本观测;
- ▶ 下线时需要撤销权限、清理订阅、保留审计和复盘记录与测试案例。

贯穿案例：一次缓存故障的真实平台旅程

1. 监控发现命中率和购物车延迟同时异常；
2. 事件服务把两个告警归并到同一服务和时间窗口；
3. CMDB 提供依赖、负责人、版本和变更关系；
4. 知识库给出热 Key、连接池和熔断的检查顺序；
5. 作业平台提出限流或回滚建议，审批后受控执行；
6. 验证平台检查错误率、队列长度和用户成功率；
7. 工单与复盘系统保存证据、结论和新规则。

观察 每一步都需要同一个 ‘incident_id’ 和同一组对象身份，否则平台只能留下零散截图。

反模式：工具都在线，但协作仍然断裂



- ▶ 告警没有对象关系，日志没有统一时间，Trace 没有业务入口；
- ▶ 工单里只有“现象”和“结论”，缺少查询、版本、证据和动作；
- ▶ 结果是个人经验被迫充当系统集成层。

平台接口设计：六件事必须说清楚

身份 service、instance、cluster、tenant、incident。

时间 发生时间、观测时间、处理时间和数据水位。

证据 查询、样本、快照、变更和来源。

责任 owner、on-call、审批人和升级路径。

动作 建议、执行、超时、回滚和验证。

结果 恢复、未恢复、误报、未知和学习项。

各模块都要明确这六项约定，调用方才能知道如何传参、处理错误和核对结果。

第一部分小结：平台要治理的是断点

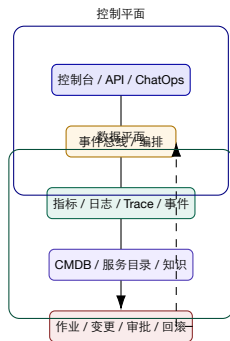
1. 监控大盘回答“看到什么”，平台还要回答“对象是谁、责任在哪、动作如何验证”；
2. 企业复杂度来自规模、异构、责任和合规的叠加；
3. 平台必须围绕对象、事件、证据和状态组织能力；
4. 第一个架构检查问题：如果删掉一个工具，事件身份和处理分工是否仍然成立？

过渡 下一部分从“为什么需要平台”进入“平台内部的数据如何流动”。

二、总体架构与数据流

把信号变成事件，把事件放进对象和流程，把动作变成可验证结果

参考架构：数据平面、控制平面与体验层



五条数据流，组成一条事故链

1. 观测流：应用、节点、GPU、模型和业务产生指标、日志、Trace；
2. 事件流：规则、检测器和人工报告产生事件并去重、聚合；
3. 上下文流：对象关系、变更、负责人、知识和历史案例补充事件；
4. 动作流：查询、建议、审批、执行、回滚和验证形成状态变化；
5. 学习流：结果、反馈、误报和复盘更新规则、Runbook 和评测集。

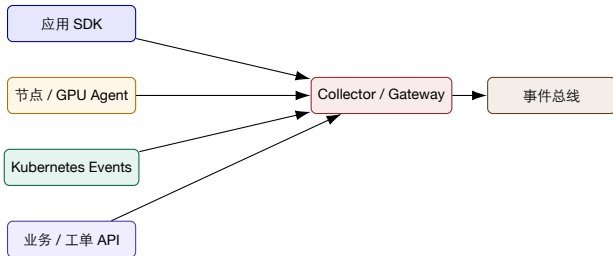
不要只画“数据进入平台”，还要画“结果回到数据和知识”。

数据平面与控制平面：为什么要分开？

维度	数据平面	控制平面
主要内容	指标、日志、Trace、事件、快照	身份、策略、工作流、审批、动作和审计
变化特征	高吞吐、可扩展、保留和查询优化	低频但高风险、强调一致性和权限
失败影响	查询不完整、延迟升高、数据丢失	错误动作、越权、状态错乱、不可回滚
设计重点	采集、路由、存储、索引和水位	状态机、幂等、审批检查、回滚和证据

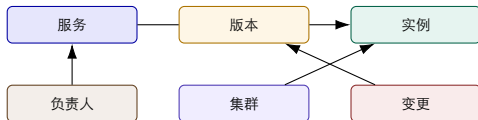
工程判断 同一个系统可以部署在一起，但数据读写和危险动作必须有不同的可靠性与安全策略。

接入层：把异构信号送进平台



- ▶ 采集器负责协议、批量、重试和限流，不负责业务结论；
- ▶ 网关负责租户、身份、采样、脱敏和路由；
- ▶ 接入规范至少包含 **schema** 版本、时间戳、来源和丢失语义。

对象模型：先认识“谁”，再解释“什么”



- ▶ CMDB/服务目录不是“资产通讯录”，而是跨系统关联的对象主键；
- ▶ 同一个服务名必须能够映射到部署版本、运行实例、责任团队和业务能力；
- ▶ 对象关系版本化，才能解释历史事件中的“当时是谁”。

事件模型：把告警变成可管理的对象

字段	示例	为什么重要
事件身份	incident_id、dedup_key	去重、协作、审计和回放
对象范围	service、cluster、region、tenant	查询边界和负责人路由
时间范围	observed_at、window_start/end	证据对齐和传播方向判断
信号摘要	metric、log、trace、change	触发条件和可解释性
状态	open、triaged、mitigated、resolved	工作流和升级规则
置信度与影响	confidence、impact、SLO	排序、沟通和操作前检查

事件 schema 可以扩展，但字段语义必须稳定；“备注”不能替代结构化状态。

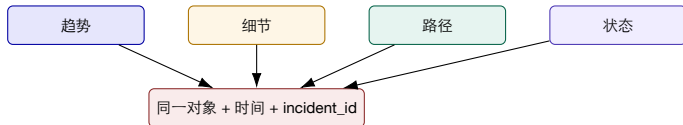
指标、日志、Trace、事件：四种信号如何互补？

指标 聚合趋势、
阈值、SLO 和容
量。

日志 异常细节、
错误消息和上下文。

Trace 一次请求
跨服务的路径和耗
时。

事件 告警、变更、
发布、工单和人工
判断。



规范化：平台不要把每个来源当成一种世界

1. 把不同来源的服务名、实例名、区域和环境映射到对象模型；
2. 把时间统一到明确的时区、精度和水位语义；
3. 把严重性、状态、错误码和动作结果映射到受控枚举；
4. 保留原始 payload、schema 版本和来源，便于回溯；
5. 对未知字段采用显式 'unknown'，不静默丢弃。

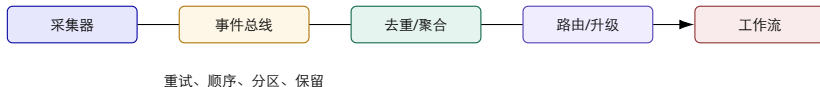
不要过度规范 统一的是跨系统最小语义；领域特有字段应保留在扩展区，并登记 owner 和版本。

丰富上下文：从“异常”变成“可行动事件”



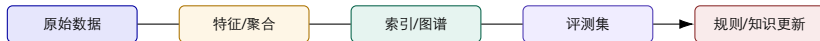
- ▶ 对象上下文来自 CMDB/服务目录；
- ▶ 依赖上下文来自拓扑、调用链和队列关系；
- ▶ 变更上下文来自发布、配置、策略和资源操作；
- ▶ 责任上下文来自团队、值班、升级和审批规则。

实时流：告警为什么需要事件总线？



- ▶ 总线让多个消费者共享事件，而不是互相调用；
- ▶ 去重、抑制、维护窗口和升级策略要有可解释记录；
- ▶ 事件处理必须幂等，重复投递不能重复执行危险动作。

批处理流：历史数据是上下文，不是垃圾场



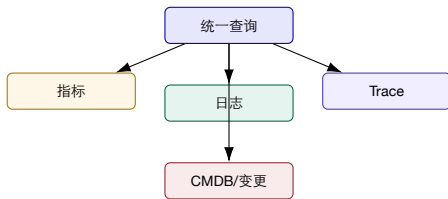
- ▶ 历史事件用于基线、聚类、容量预测、回放和评测；
- ▶ 原始数据、清洗数据、派生特征和人工标签要分层保留；
- ▶ 每次规则或知识更新都应记录训练/检索样本和版本。

存储分层：热数据、温数据和证据归档

层级	典型内容	访问模式	设计关注
热层	最近指标、活动事件、当前 Trace	秒级查询和工作流触发	延迟、容量、降采样
温层	历史日志、工单、变更、知识	分钟级检索和案例分析	索引、成本、权限
冷层	原始 payload、审计、回放样本	低频、合规和复盘	完整性、保留、可导出

平台取舍 “全部永久保留” 既昂贵又不等于可用；保留策略应按故障价值、合规和评测用途设计。

查询联邦：用户要的是一条证据路径



返回统一对象、时间窗口、证据链接和查询状态

- ▶ 联邦查询不是把所有数据复制到一个库；
- ▶ 平台负责身份解析、时间对齐、超时和结果合并；
- ▶ 部分数据源不可用时，结果必须显式标记“不完整”。

数据质量本身也要被观测

新鲜度

数据到达延迟

完整性

字段和样本覆盖

一致性

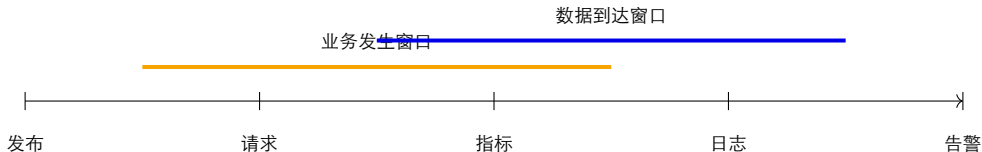
对象和状态对齐

可用性

查询和服务成功率

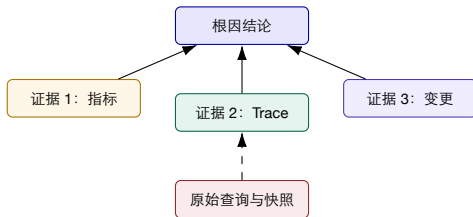
- ▶ 业务事件如果延迟 20 分钟到达，模型再准确也无法帮助值班；
- ▶ Trace 采样率、日志脱敏和 CMDB 漂移会改变结论可信度；
- ▶ 平台应把数据质量作为 SLO，并在质量下降时自动降级。

时间水位：跨源关联最容易被忽略的字段



- ▶ 'occurred_at' 和 'observed_at' 不是同一个时间；
- ▶ 延迟、时钟漂移、批量写入和采样会造成假因果；
- ▶ 事件页面应显示水位和缺口，让使用者知道结论是否可能被迟到数据改变。

数据血缘：每个结论都应该能“点回去”



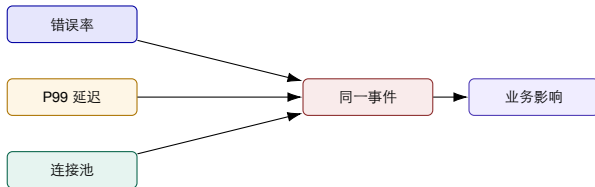
证据要求 结论的可解释性不是多写一段自然语言，而是保留来源、查询、时间、样本和缺失信息。

告警路由：把“严重”转成“谁现在要做什么”

1. 按租户、服务、环境和维护窗口过滤；
2. 按业务影响、SLO、置信度和历史模式排序；
3. 路由给值班团队、领域专家和沟通角色；
4. 生成事件、工单或演练任务，避免只发通知；
5. 超时升级，重复事件合并，恢复事件自动关闭但保留证据。

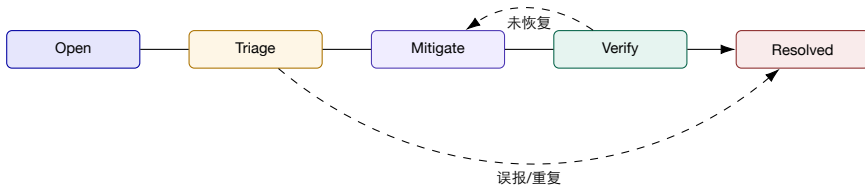
路由策略应版本化；否则“为什么这次通知了 A 而不是 B”无法复盘。

事件关联：去重只是第一步



- ▶ 去重：相同来源、对象和键只保留一个事件；
- ▶ 聚合：多个信号在时间和拓扑上形成同一故障假设；
- ▶ 关联：把事件映射到业务影响、变更和责任；
- ▶ 反关联：记录为何没有合并，避免错误吞掉独立故障。

事件状态机：消息不是流程



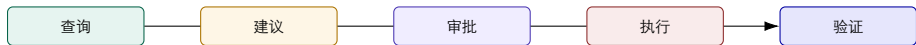
- ▶ 每个状态都有进入条件、负责角色、超时和退出证据；
- ▶ 状态转换幂等，重复消息不能让事件跳过审批；
- ▶ UI、API、工单和 ChatOps 只是状态机的不同入口。

知识流：文档、案例、规则和 **Runbook** 各司其职

资产	适合回答的问题	需要的治理
文档	系统如何设计、约束是什么？	owner、版本、适用范围、过期标记
案例	过去发生过什么，如何验证？	时间、证据、结论、反例和复盘人
规则	哪些信号满足什么条件？	版本、覆盖、误报、回滚和灰度
Runbook	下一步按什么顺序操作？	权限、前置条件、超时、回滚和验收

第 12 讲会继续讨论大模型如何使用这些资产；本讲先定义资产的接口和生命周期。

权限与策略：查询可以自动化，执行必须分级



- ▶ 查询权限限定数据范围、字段和时间窗口；
- ▶ 建议权限限定可读的 Runbook、工具和策略；
- ▶ 执行权限要求对象、理由、审批、超时和回滚；
- ▶ 高风险动作应支持人工接管和紧急停止。

审计与回放：工程团队如何证明“发生了什么”？

审计记录

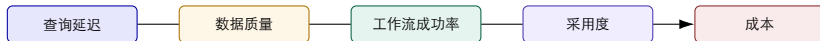
- ▶ 谁在什么租户、什么事件上做了什么；
- ▶ 使用了哪个工具、版本、参数和策略；
- ▶ 结果、错误、审批和人工修改是什么。

回放记录

- ▶ 输入信号和时间窗口是否可重建；
- ▶ 查询与动作是否可模拟而不触碰生产；
- ▶ 新规则、新知识和新工作流是否改善结果。

平台价值 没有审计，安全团队不敢放权；没有回放，工程团队无法迭代。

平台也需要自己的可观测性



- ▶ 平台故障不能只用平台自己来观测，应保留独立健康信号；
- ▶ 要区分“数据源坏了”“查询层慢了”“策略没有命中”“用户不用了”；
- ▶ 自观测是后续规模化和成本治理的基础。

案例回放：缓存故障在平台上怎样被看见？



1. 左侧告警进入统一事件；
2. 指标、日志、Trace 和变更在同一事件上下文展开；
3. 诊断结论带证据和置信度，不只输出一段摘要；
4. 动作必须经过平台策略和人工接管；
5. 结果回写事件、工单和知识资产。

第二部分小结：数据流的五个检查点

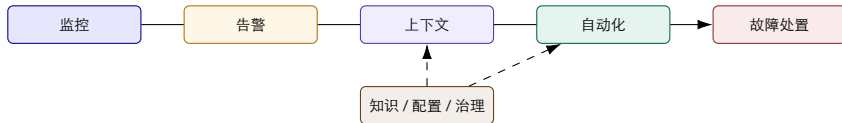
1. 是否有稳定的对象身份？
2. 是否区分发生时间、到达时间和数据水位？
3. 是否能把信号关联成事件并进入状态机？
4. 是否能把查询、动作和验证写成可审计证据？
5. 数据或工具不可用时，是否能显式降级而不是假装确定？

过渡 下一部分把这些数据流映射为六类平台能力，并定义它们之间的接口。

三、能力分层与接口约定

监控、告警、自动化、配置、知识和处置不是六个孤岛

能力分层：从“看见”到“改变”



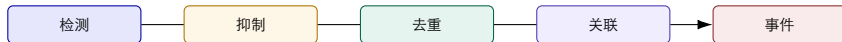
上下文、知识和治理是横切能力；它们不是流程最后再补的附件。

监控层：从单指标到服务目标

- ▶ 资源监控：CPU、内存、磁盘、网络、GPU 和队列；
- ▶ 服务监控：吞吐、错误率、延迟、饱和度和依赖健康；
- ▶ 业务监控：订单成功率、支付转化、模型质量和用户体验；
- ▶ 变更监控：发布、配置、策略、扩缩容和权限变化；
- ▶ 平台监控：采集延迟、查询失败、规则命中、动作成功率和成本。

关键原则 监控指标必须能映射到对象、SLO 和动作；“有很多指标”不等于“可诊断”。

告警层：从噪声管理到事件管理



- ▶ 告警是信号，事件是需要责任人和状态管理的对象；
- ▶ 去噪指标不能只看告警数量，还要看漏报、重复、响应和恢复质量；
- ▶ 维护窗口和静默策略必须可审计，避免把问题藏起来。

自动化层：Runbook 不是命令清单

前置条件 对象、权限、版本、维护窗口。

步骤 查询、判断、动作、等待、验证。

失败语义 超时、重试、部分成功、人工接管。

安全出口 审批、限流、回滚、停止和审计。

动作的“反面”同样要被设计：不满足条件时，系统应该怎样安全地不做。

配置管理层：状态、意图和变更必须对齐



- ▶ CMDB 记录“对象是什么”，配置系统记录“应该怎样运行”；
- ▶ 差异检测、审批、灰度和回滚把配置变化纳入故障链；
- ▶ 变更窗口和实际生效时间是诊断的重要证据。

知识层：把经验变成可复用、可复审资产

1. 采集：故障工单、复盘、Runbook、架构文档和专家标注；
2. 整理：对象、症状、证据、动作、结果和适用范围结构化；
3. 检索：按服务、版本、环境、时间和故障模式筛选；
4. 使用：为人、规则、工作流和后续智能体提供上下文；
5. 复审：发现过期、冲突、低覆盖和错误建议后更新或下线。

知识资产如果没有 owner、版本和反例，越多越可能增加误导。

故障处置层：从告警到恢复验证



- ▶ 定界回答“影响谁”，假设回答“为什么”；
- ▶ 计划把证据查询和动作顺序写出来；
- ▶ 验证必须同时看技术信号、业务 SLO 和副作用；
- ▶ RCA Agent 的详细实现留到第 13 讲，本页只定义平台接口。

容量与 SLO：平台不能只在事故时有用

被管系统 SLO

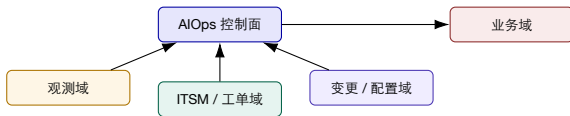
- ▶ 可用性、延迟、错误率和业务成功率；
- ▶ GPU 利用率、队列、Token 吞吐和模型质量；
- ▶ 变更失败率和恢复时间。

平台自身 SLO

- ▶ 数据新鲜度、查询延迟和事件处理延迟；
- ▶ workflow 成功率、误报率和人工接管率；
- ▶ 审计完整性、隔离正确率和成本上限。

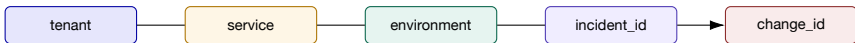
注意 如果平台自身不稳定，值班人员会在最需要它时绕开它。

集成层：平台不应吞掉所有领域系统



- ▶ 平台统一事件身份、权限、状态和证据入口；
- ▶ 观测、作业、配置和业务系统保留专业能力与数据主权；
- ▶ 集成靠稳定 API、事件、Webhook 和适配器，不靠复制数据库。

共享标识：平台协同的最小主键集合



- ▶ ‘tenant’ 限定数据和动作边界；
- ▶ ‘service’/‘environment’ 连接指标、日志、Trace、CMDB 和负责人；
- ▶ ‘incident_id’ 连接事件、工单、 workflow、审计和复盘；
- ▶ ‘change_id’ 连接发布、配置、策略和回滚。

平台接口约定：每个能力都要回答五个问题

1. 输入 **schema** 是什么，字段是否有版本？
2. 成功输出是什么，如何判断部分成功？
3. 错误、超时、重试和幂等语义是什么？
4. 需要什么权限、审批和租户上下文？
5. 如何提供证据、指标、日志、**Trace** 和审计？

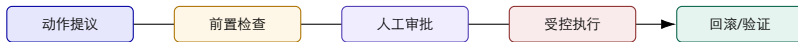
动作说明 请把“重启服务”改写成一个平台动作说明，而不是一句命令。

Tool Registry: 让工具可发现、可评估、可淘汰

- ▶ 名称、用途、版本和 owner;
- ▶ 输入、输出、错误和超时;
- ▶ 支持的租户、集群、环境和权限;
- ▶ 只读、建议、执行三级风险。
- ▶ 成功率、延迟、覆盖率和成本;
- ▶ 真实案例和合成回放成绩;
- ▶ 失败模式、回滚方式和停用条件;
- ▶ 与其他工具的依赖和替代关系。

工具治理 工具接入平台前要验收，接入后要记录版本、监测失败，并支持停用或替换。

ActionStack: 把动作从脚本升级为受控执行



- ▶ 每个动作都有风险等级和最小权限；
- ▶ 执行前后采集对象状态，支持差异比较；
- ▶ 失败时优先回到安全状态，而不是继续尝试更多命令。

评测层：平台能力要用结果衡量

评测对象	核心指标	反例与边界
检测 关联 诊断 动作 平台	召回、误报、提前量 去重率、聚合准确率、漏并率 根因命中、证据覆盖、置信度校准 成功率、恢复时间、副作用 查询延迟、采用度、成本、审计完整性	低频故障和分布变化 多故障并发、时间延迟 证据缺失时不要强行给结论 权限、人工接管和回滚 业务价值不能只看点击量

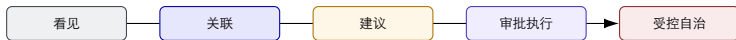
“自动化率” 如果没有安全边界和恢复质量，可能只是把风险转移给平台。

平台工作台：把证据放在同一个事件上下文



- ▶ 拓扑负责回答“可能影响哪些对象”；
- ▶ 指标、日志、Trace 负责回答“发生了什么”；
- ▶ 工单、变更和知识负责回答“谁做过什么”；
- ▶ 工作流和审计负责回答“接下来如何安全行动”。

能力成熟度：先把链路接通，再追求自治



- ▶ 每一级都要有数据质量、流程证据和停止条件；
- ▶ 自治不是平台的默认终点，某些动作永远需要人工决策；
- ▶ 评价成熟度时同时看覆盖、可靠性、风险和组织采用。

统一建设、域内建设和外部集成：怎样划边界？

适合平台统一	适合业务域保留	适合外部系统集成
事件身份、权限、审计、工作流、工具目录、评测、租户边界	业务语义、领域指标、专业 Runbook、领域知识、责任判断	指标/日志/Trace 存储、CMDB、ITSM、作业、发布、消息和云 API

边界原则 平台统一最小共同语言；领域保留专业判断；外部系统通过稳定接口约定接入。

四个工程反模式

大一统 把所有数据复制到一个平台，成本和责任都失控。

自动化先行 没有对象、权限和验证就开放执行。

知识堆积 文档越多，版本、适用范围和冲突越不清楚。

只看点击 用户点击很多，不代表事件恢复更快或更安全。

反模式的共同原因：把平台当作产品页面，而不是责任、状态和证据系统。

试点边界：一条链路、一个域、一个责任人

1. 选一类高频、低风险、可回放的事件；
2. 只覆盖一个业务域和明确的服务集合；
3. 定义事件身份、数据水位、查询工具和 Runbook；
4. 先做只读和建议，逐步开放审批执行；
5. 规定数据不可用、证据冲突和动作失败时的降级路径。

阶段验收 验收试点时，应逐项检查谁接收事件、谁批准操作、谁执行，以及谁确认恢复。

第三部分小结：能力分层就是接口设计

1. 监控提供信号，告警把信号转成事件；
2. CMDB、配置、变更和知识补齐上下文；
3. 工具、工作流和动作平台把建议变成受控行动；
4. 评测、审计和平台自观测决定能力能否规模化；
5. 统一平台接口约定，保留领域语义，是平衡复用与自治的关键。

算法接口：从信号到事件

平台的每一层都对应可解释的算法问题：检测、去重、关联、排序与操作前检查

算法一：异常检测先建立“正常基线”

稳健基线（示例公式）

$$\hat{x}_t = \text{median}(x_{t-w}, \dots, x_{t-1})$$

$$r_t = x_t - \hat{x}_t, \quad s_t = \frac{|r_t|}{1.4826 \text{ MAD}_t + \varepsilon}$$

若 $s_t > \tau$ 连续 k 次，则产生候选异常。

- ▶ 中位数和 MAD 可抵抗少量离群污染；缺失值须另行处理；
- ▶ w 控制“看多长的过去”， τ 控制灵敏度；
- ▶ 连续 k 次或区间占比能减少单点噪声；
- ▶ 候选异常还要经过对象、维护窗口和数据质量检查。

公式是平台入门级基线，不代表所有生产场景；复杂季节性和多变量关系需要更强模型。

案例：缓存命中率的异常分数怎样解释？

时间	观测 x_t	基线 $\hat{x}_t$	s_t	解释
09:00	0.991	0.990	0.2	正常波动
09:01	0.989	0.990	0.2	正常波动
09:02	0.973	0.990	3.4	候选异常
09:03	0.941	0.989	9.6	连续异常，升级候选
09:04	0.938	0.987	9.8	结合连接池与发布事件

算法输出 不是“根因是缓存”，而是“缓存命中率偏离基线，置信度随连续性上升”。

平台动作 把候选异常送入关联层，等待其它信号和上下文共同判断。

假设各时刻 $1.4826 MAD + \varepsilon = 0.005$ ；取 $\tau = 3, k = 2$ ，09:03 首次满足连续两点越阈。

异常检测算法怎么选：静态、时序还是多变量？

场景	简单基线	更强方法	平台注意点
静态阈值/资源上限	分位数、MAD、EWMA	规则与分段阈值	易解释，需维护阈值和窗口
季节性单变量	STL、季节分位数、CUSUM	时序 Transformer	关注节假日、流量和数据延迟
多变量耦合	相关矩阵、PCA	TranAD、Anomaly former	需要训练数据、漂移监控和回放
同伴比较	分组基线、相似性分数	Minder 类同伴检测	需要稳定分组和正常样本

工程取舍 先用可解释基线建立数据质量和告警要求，再逐步引入复杂模型；否则模型错误很难区分是算法问题还是数据问题。

算法二：告警去重的核心是稳定键，而不是字符串相似

去重键（示例定义）

$key = H(\text{tenant}, \text{object}, \text{rule}, \text{dimensions})$

1. 规范化服务名、环境、区域和标签；
2. 移除不会改变事件身份的动态字段；
3. 在时间窗口内合并相同 key；
4. 记录被合并告警的数量和来源。

- ▶ 同一服务的 100 个实例告警，不一定是 100 个事故；
- ▶ 不同版本、不同租户或不同故障域不能盲目合并；
- ▶ 去重结果需要可反查，不能只留下一个摘要；
- ▶ 合并失败也要记录原因，支持后续调参。

去重解决“重复通知”；它还没有回答“这些信号是否属于同一个根因”。

算法三：告警聚合把时间、拓扑和变更合成边权

候选关联边（方法概括）

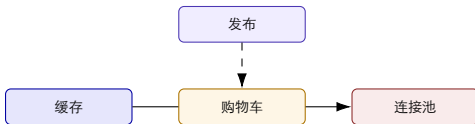
$$w_{ij} = \alpha e^{-\Delta t_{ij}/\tau} + \beta \mathbf{1}[(i,j) \in E] + \gamma \mathbf{1}[\text{change}(i,j)] + \delta \text{sim}(\text{tags}_i, \text{tags}_j)$$

- ▶ Δt 越小，时间上越可能属于同一波故障；
- ▶ E 是服务/资源拓扑边，表示传播路径或依赖关系；
- ▶ 变更项把发布、配置、扩缩容作为额外证据；
- ▶ 标签相似度只能辅助聚合，不能单独证明因果。

算法输出 对边权设阈值或做社区发现，得到事件簇；再把事件簇交给排序和诊断，而不是直接当作根因。

案例：四条告警如何聚成一个事件？

信号	证据摘要	关联分数	平台解释
a_1	缓存命中率下降, 09:02	0.88	与购物车服务同对象、时间接近
a_2	购物车 P99 上升, 09:03	0.82	与 a_1 有调用链边
a_3	DB 连接池耗尽, 09:03	0.74	与购物车依赖边相连
a_4	配置发布, 09:01	0.69	时间先于异常, 作为变更证据



聚合结果是“同一事件的候选结构”；还要用反证和恢复验证排除错误合并。

算法四：事件排序要显式组合影响、紧迫性和可信度

优先级分数（示例定义）

$$P(e) = w_I I(e) + w_U U(e) + w_C C(e) + w_F F(e) - w_R R(e)$$

- ▶ I : 业务影响、受影响用户和 SLO;
- ▶ U : 增长速度、剩余预算和升级时限;
- ▶ C : 检测和关联的置信度;
- ▶ F : 证据新鲜度和完整性;
- ▶ R : 数据缺失、冲突和误报风险。
- ▶ 高影响但低可信度的事件应优先人工核查，而不是自动执行;
- ▶ 低影响但高可信度的事件可以进入批量治理队列;
- ▶ 权重按业务域和阶段验收配置，不能写死在页面代码里。

算法五：根因候选排序如何进入平台，而不越权下结论？

候选节点评分（方法概括）

$$R(v) = \lambda_1 E(v) + \lambda_2 T(v) + \lambda_3 G(v) + \lambda_4 C(v) - \lambda_5 X(v)$$

- ▶ $E(v)$ ：证据覆盖，能解释多少指标、日志和 Trace；
- ▶ $T(v)$ ：时间先后，候选是否早于症状出现；
- ▶ $G(v)$ ：拓扑/调用链距离，是否位于传播路径上；
- ▶ $C(v)$ ：变更、配置或历史案例支持程度；
- ▶ $X(v)$ ：反证、缺失数据和互相冲突的证据惩罚。

平台输出 返回候选列表、证据链接、置信度和缺口；“第一名”仍需人或验证动作确认。

算法六：动作策略是一个可验证的布尔判定

动作放行条件（示例定义）

$$\text{allow}(\mathbf{a}) = \text{scope} \wedge \text{fresh} \wedge \text{owner} \wedge \text{approval} \wedge \text{rollback}$$

1. 目标是否属于当前租户、集群和服务作用域？
2. 证据是否新鲜，数据质量是否达到门槛？
3. 当前值班和审批角色是否有效？
4. 是否存在可执行的回滚和恢复验证？
5. 任一条件不满足，动作降级为建议或人工接管。

这一步不是模型推理，而是策略引擎和工作流的确定性检查。

算法分数如何推动事件状态机？

状态	触发算法输出	允许的下一步	必须保留的证据
Open Triage Mitigate Verify	异常分数超过阈值 事件簇和优先级已计算 策略判定为可建议/可执行 技术和业务信号重新计算	去重、聚合、补上下文 指派、查询、生成假设 审批、动作或人工接管 关闭、回滚或升级	原始信号、窗口、数据质量 关联边、影响、负责人 前置检查、版本、权限 前后对照、SLO、回滚结果

关键点 算法输出只有在进入状态机后才成为工程行为；状态机又反过来约束算法不能越权。

算法边界：什么时候必须降级？

数据缺失 关键指标、日志或 Trace 迟到/断流。

证据冲突 不同来源对对象、时间或状态给出矛盾结论。

分布变化 发布、流量、拓扑或指标语义已改变。

动作不可逆 没有可靠回滚、审批或副作用验证。

- ▶ 算法返回“未知/证据不足”比返回一个漂亮的错误答案更安全；
- ▶ 降级路径：只读查询、扩大人工接管、延迟执行、建立复盘任务；
- ▶ 每次降级都计数，成为平台数据质量和工程改进指标。

四、蓝鲸式工程实践

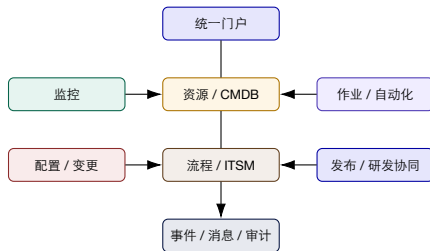
用开源平台的模块化思路，理解企业 AIOps 如何实施

为什么用蓝鲸作为工程实践案例？

- ▶ 它适合讨论“平台由多个专业模块组成”的工程思想；
- ▶ 能把资源、监控、作业、配置、流程和服务目录放在同一套平台化语境下；
- ▶ 开源组件可用于分析扩展、适配、权限、部署和升级。
- ▶ 本页及后续不依赖某个具体版本的产品承诺；
- ▶ 重点是模块边界、事件格式和工程治理；
- ▶ 企业实际使用仍需核对社区文档、许可证、插件成熟度和自身环境。

“蓝鲸式”在本讲中指模块化、插件化、流程化和工程治理的案例视角。

蓝鲸式平台模块地图（简化模型）

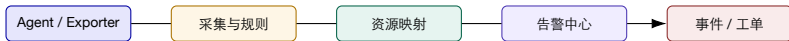


资源模型：CMDB 让不同模块说同一种“对象语言”

1. 定义主机、容器、集群、服务、进程、数据库、队列等资源类型；
2. 定义资源之间的包含、依赖、部署、调用和归属关系；
3. 为资源记录环境、区域、标签、负责人和生命周期；
4. 让监控、作业、配置和工单都引用同一对象身份；
5. 记录关系变更，支持历史事件回看。

案例 没有可靠资源模型，监控告警只能指向 IP，自动化也不知道自己会影响什么。

监控集成：从采集到对象化告警



- ▶ 采集系统负责原始数据和检测；
- ▶ CMDB 负责把告警映射到资源、服务和负责人；
- ▶ 告警中心负责抑制、去重、升级和事件化；
- ▶ 平台 workflow 负责后续协作，不把监控系统变成万能控制器。

告警工程：蓝鲸式平台关注“告警到人”的完整路径

1. 规则定义：阈值、同比、基线、异常检测和维护窗口；
2. 告警标准化：对象、级别、摘要、开始时间、恢复时间；
3. 告警聚合：按资源、服务、拓扑和时间窗口关联；
4. 告警分派：值班、领域团队、升级路径和通知渠道；
5. 告警处理步骤：确认、处置、检查恢复、标记误报、复盘和修改规则。

告警中心的关键产品不是“更多通知渠道”，而是状态、责任和反馈。

作业平台：把脚本变成可治理的动作资产

脚本时代

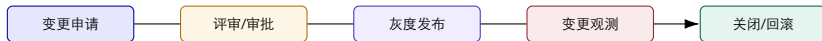
- ▶ 个人电脑和跳板机上散落脚本；
- ▶ 参数、权限、输出和失败语义不一致；
- ▶ 事故后很难证明到底执行过什么。

平台化作业

- ▶ 作业模板、参数校验和目标资源；
- ▶ 审批、并发、超时、重试、回滚；
- ▶ 日志、结果、审计和可复用编排。

课程连接 第 9 讲的 Tool Registry 解决“发现和评估”，平台作业解决“安全运行和审计”。

配置与变更：把“谁改了什么”纳入故障证据



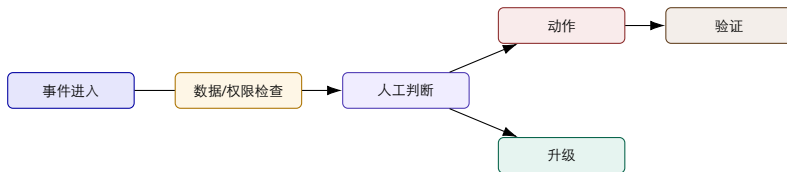
- ▶ 变更系统提供时间、对象、版本、操作者和审批证据；
- ▶ AIOps 平台把异常信号与变更窗口关联，而不是把变更当作备注；
- ▶ 恢复后比较变更前后状态，并把结果回写到发布质量。

流程与 ITSM：工单是状态承载，不是聊天记录



- ▶ 工单引用事件、对象、证据、动作和结果；
- ▶ 任务分解、升级和 SLA 由状态机驱动；
- ▶ 复盘把工单变成知识、规则、Runbook 和评测样本。

流程编排：平台如何表达“条件分支”和“人工接管”？



工作流的价值在于显式表达等待、分支、超时和接管，而不只是把步骤画成直线。

插件化与事件总线：让平台可以长大

- ▶ 新的数据源通过采集插件或适配器接入；
- ▶ 新的动作通过工具/作业插件注册；
- ▶ 新的流程通过编排模板复用；
- ▶ 新的视图通过 API 和事件订阅扩展；
- ▶ 事件总线解耦生产者和消费者，减少模块之间的硬编码依赖。

升级原则 插件隔离、版本兼容、灰度发布和可回退，比“能否快速加一个按钮”更重要。

多租户工程：共享平台，不共享风险

隔离对象	需要隔离的内容	典型控制
数据 资源 权限 成本	指标、日志、Trace、工单和知识 集群、主机、服务和动作目标 查询、建议、执行和审批 存储、查询、模型和执行资源	namespace、标签、查询策略、脱敏 资源域、作用域、目标校验 RBAC/ABAC、最小权限、临时授权 配额、计量、预算、限流

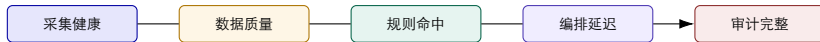
租户隔离同时影响安全、计费、数据质量和模型上下文，不能只做 UI 过滤。

高可用与灾备：平台故障不能让事故更糟

- ▶ 采集和事件入口可重试、可缓冲；
- ▶ 关键状态持久化，重复消息幂等；
- ▶ 控制面与数据面故障隔离；
- ▶ 危险动作支持全局停止。
- ▶ 只读查询可以降级到缓存或源系统；
- ▶ 自动化不可用时回退人工流程；
- ▶ 审计和事件状态需要独立备份；
- ▶ 灾备演练要验证恢复后的状态一致性。

安全底线 宁可少做动作，也不能在上下文不完整时重复执行高风险动作。

平台自监控：谁来监控 AIOps 平台？



- ▶ 看入口延迟、丢失率、队列堆积和消费者 lag;
- ▶ 看查询成功率、证据覆盖、工具失败和工作流超时;
- ▶ 看采用度、人工绕过率和成本，识别“平台被架空”。

安全工程：把越权从“事后发现”前移到“动作前检查”

1. 身份认证：人、服务、机器人和临时授权；
2. 资源授权：租户、集群、环境、服务和字段范围；
3. 动作授权：只读、建议、执行、回滚和紧急停止；
4. 数据治理：脱敏、留存、导出、加密和访问审计；
5. 供应链治理：插件、脚本、模型和知识来源可追踪。

平台开放执行能力前，必须具备权限检查、操作记录和回滚路径。

部署与升级：平台工程是长期运维对象

部署检查

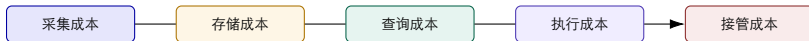
- ▶ 依赖服务、密钥、网络和资源配额；
- ▶ 数据迁移、索引、队列和备份；
- ▶ 观测、告警、回滚和故障演练。

升级检查

- ▶ API/schema/插件兼容性；
- ▶ 灰度、双写、回退和数据校验；
- ▶ 旧 workflows、权限和审计继续可读。

工程结论 能升级、能回退、能读历史，才称得上企业平台。

成本治理：平台的每一条数据流都有价格



- ▶ 标签、采样、保留、索引和查询范围决定观测成本；
- ▶ 自动化失败会把节省的人工成本重新变成事故成本；
- ▶ 评测应把恢复收益、误动作损失、模型调用和人工接管放在一张表中。

蓝鲸式平台案例：支付服务的告警到恢复

1. 监控模块发现支付成功率下降并产生告警；
2. 资源模型把告警关联到支付服务、版本、节点和负责人；
3. 告警中心聚接口约定窗口的数据库连接池和网关延迟信号；
4. 流程模块创建事件和工单，作业模块提供只读检查；
5. 变更模块发现 10 分钟前的配置发布；
6. 经审批后执行限流/回滚，平台验证业务 SLO；
7. 工单关闭时把证据、结论和新 Runbook 写回知识库。

案例是简化模型：目的是看见模块协同和状态流，不是复述某个厂商的交付承诺。

第四部分小结：开源平台给我们的工程启发

1. 资源模型是跨模块共同语言；
2. 监控、告警、作业、配置、流程和知识通过事件和对象连接；
3. 插件化让平台扩展，接口约定和版本让平台可升级；
4. 多租户、权限、审计、高可用和成本不是上线后的补丁；
5. 企业采用开源平台时，必须做版本、许可证、插件成熟度和责任边界评估。

五、综合案例与平台蓝图

用一条故障链把架构、能力和工程治理串起来

综合案例：电商客户的平台蓝图



- ▶ 先覆盖缓存、连接池、消息堆积和购物车四类高频问题；
- ▶ 把发布、配置和扩容作为一级证据源；
- ▶ 第一阶段只读与建议，第二阶段审批执行，第三阶段再评估自治。

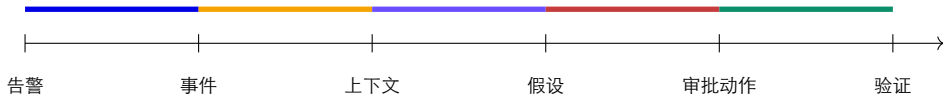
事件信封：让一次事件能够跨系统旅行

事件信封（示例）

`incident_id`、`tenant`、`service`、`environment`、`severity`、`occurred_at`、`observed_at`、`window`、`signals[]`、`changes[]`、`owner`、`state`、`confidence`、`evidence[]`、`actions[]`。

- ▶ 监控写入 `'signals[]'`，CMDB 写入对象和 `owner`，变更系统写入 `'changes[]'`；
- ▶ 工作流更新 `'state'`、`'actions[]'` 和审计；
- ▶ 知识和评测读取完整事件，并在复盘后回写版本化资产。

综合案例时间线：平台怎样缩短“找数据”的时间？



- ▶ 没有平台时，1.7–5.1 的上下文拼接主要由人完成；
- ▶ 有平台时，仍需人判断，但数据入口、证据和责任已被预先组织；
- ▶ 验收不只看“有没有自动回复”，还看跨系统搬运时间和证据完整性。

前后对比：平台价值不等于自动化率

观察项	平台前	平台后（目标状态）
上下文获取	人工切换多个系统	事件上下文统一入口
责任分派	群聊和经验	服务目录、值班和升级规则
动作执行	临时脚本和口头确认	作业说明、审批、回滚和验证
记录复盘结果	会议纪要和个人笔记	事件、证据、Runbook、规则和评测集
平台改进	只修业务系统	同时观测数据质量、工具质量和采用度

“平台后” 仍可能需要人工；区别是人工把注意力放在判断和决策，而不是搬运上下文。

90 天路线图：从对象和事件开始

0-30 天 选域、对象、事件 schema、数据质量基线、只读查询。

31-60 天 告警聚合、CMDB 关联、Runbook、工单和审计。

61-90 天 审批执行、回滚验证、评测回放、成本和采用看板。

1. 每个阶段有明确退出条件；
2. 不满足数据或安全门槛时，停在只读/建议模式；
3. 90 天结束后决定扩域、重构或停止，而不是默认续期。

平台验收：至少要看五类证据

1. 覆盖证据：目标服务、事件、数据源和 Runbook 覆盖率；
2. 质量证据：数据新鲜度、告警误报、事件关联和证据完整性；
3. 效率证据：上下文获取时间、MTTA、MTTR 和人工切换次数；
4. 安全证据：权限命中、审批、回滚、越权阻断和审计完整性；
5. 采用证据：使用率、绕过率、反馈、复用和资产更新频率。

验收原则 没有基线、对照组和停止条件，平台的“效果”只能停留在演示。

设计一个最小 AIOps 平台

1. 选择一个域：电商、金融、通信、制造或模型服务；
2. 画出 5 层上下文和 5 条数据流；
3. 定义事件信封中的 10 个必填字段；
4. 选择 3 个只读工具、1 个建议动作和 1 个审批动作；
5. 写出数据不可用、权限不足和动作失败时的降级路径。

每组 10 分钟设计，3 分钟讲解；重点看边界、接口和验证，不比拼组件数量。

方案评估标准

维度	通过标准	常见失分点
对象与事件 能力协同	有稳定身份、时间、水位和状态 监控、告警、CMDB、知识、作业、工单互相引用	只画指标和箭头 每个模块独立成岛
安全与治理 工程可行性 验收设计	查询/建议/执行分级，有审批、回滚和审计 有插件、版本、部署、升级和高可用考虑 有基线、指标、回放和停止条件	直接开放自动执行 只有理想架构，没有运维 只写“提高效率”

风险登记：平台项目最容易忽略什么？

数据漂移 对象、
标签、时间和指标
语义逐渐变化。

责任漂移 服务
owner、值班和审
批路径没有同步。

动作漂移 脚本、
权限、版本和回滚
条件变化。

采用漂移 平台
上线后用户回到
群聊和个人脚本。

治理动作 每月审查对象质量、工具质量、 workflow 结果、权限和采用度；低质量资产要下线，而不是无限堆积。

架构设计题：你会怎样评审一张平台图？

1. 先找对象、事件、状态和责任，而不是先数微服务；
2. 再看数据平面和控制平面是否分离；
3. 检查每条跨系统边的 schema、时间、错误和权限；
4. 检查动作前后是否有审批、幂等、回滚和验证；
5. 最后问：数据源、平台本身和组织采用如何被观测？

架构图的好坏，不在于画得多复杂，而在于能否支撑一次真实事件的复盘。

六、研究拓展：AI 系统故障 诊断

从训练集群、告警生命周期到 LLM 推理跨层调用链，论文怎样把问题变成可验证系统

论文阅读方法：不要只记模型名字

1. 故障对象：机器、服务、告警、请求、GPU kernel 还是变更？
2. 操作决策：检测异常、找候选、排序、解释、处置还是恢复验证？
3. 证据机制：同伴、时间、拓扑、Trace、日志、变更和干预如何组合？
4. 工程边界：采集开销、延迟、标签、漂移、权限和回滚怎样处理？

NSDI/OSDI：系统机制与生产约束

EuroSys/ASPLOS：运行时、硬件与跨层协同

FSE/ASE：软件故障与运维流程

本讲优先展开课程目录中已核对全文或原始材料的论文；后续可继续补充 OSDI、EuroSys、ASPLOS 的最新系统论文。

NSDI 2025 Minder: 训练集群中的“坏机器”先于任务失败被发现

问题

- ▶ 一个分布式训练任务跨越上千台机器；
- ▶ 一台 GPU、PCIe、NIC、存储或 CUDA/NCCL 异常机器可能拖停全任务；
- ▶ 全局阈值无法适应不同训练任务和并行角色。

论文主张

- ▶ 在同构并行任务中，用同伴机器构造动态基线；
- ▶ 每指标去噪，再做机器间相似性比较；
- ▶ 用持续性和指标优先序降低误报与检测延迟；
- ▶ 输出是异常机器，后续 RCA 仍需要跨层证据。

平台连接 AIOps 平台要把“任务角色、机器同伴、指标窗口和故障事件”作为同一对象上下文。

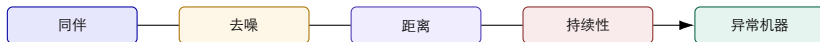
Minder 的算法拆解：四个设计互相补位

Similarity 和同一任务的正常同伴比较，而不是和固定阈值比较。

Continuity 连续多个窗口偏离才提升为异常机器。

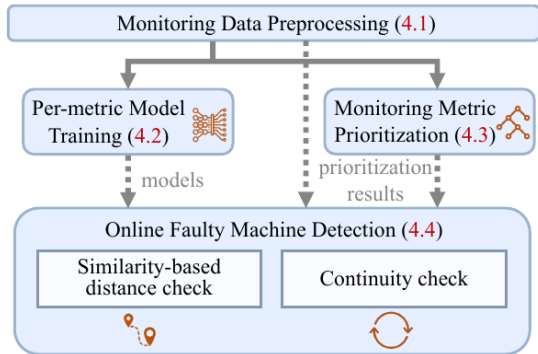
Denoising 每个指标单独去噪，避免噪声维度掩盖敏感信号。

Prioritization 优先运行最可能提前暴露故障的指标。



算法链的作用：每一步都减少一种不确定性，但没有一步单独等于根因诊断。

Minder 系统机制：训练和在线检测要拆开



1. 对齐、补齐和归一化；
2. 每指标训练去噪模型；
3. 学习指标优先序；
4. 在线计算同伴距离；
5. 检查异常持续性；
6. 输出异常机器和证据窗口。

Minder 的实验数字应该怎样读？

3.6 s

论文报告的平均反应时间

0.904

论文报告的 Precision

0.893

论文报告的 F1

- ▶ 这些数字依赖采样周期、窗口、同伴分组和生产标签；
- ▶ 论文主要解决异常机器发现，不等于完整故障解释或自动恢复；
- ▶ 迁移到推理集群时，需要重新验证异构 GPU、并行角色和共同上游故障；
- ▶ 平台验收应补充数据新鲜度、人工接管和副作用指标。

TELLER: LLM 推理故障必须跨过“服务层”下钻到执行层

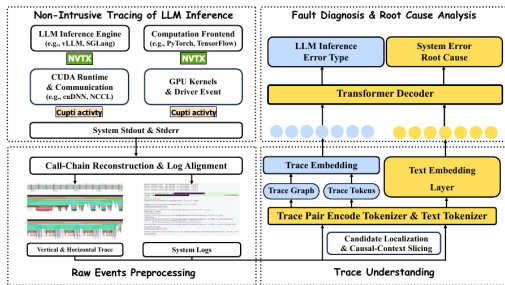
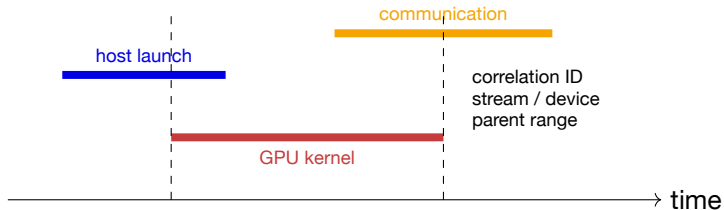


Figure 1: Overview of TELLER. TELLER first performs non-intrusive cross-layer tracing of LLM inference, then reconstructs

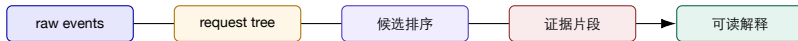
- ▶ 请求级异常可能来自 queue、prefill、KV cache、GPU kernel、通信或调度；
- ▶ 服务日志只记录“请求超时”，无法解释跨层传播；
- ▶ TELLER 用非侵入式观测重建 per-request call-chain；
- ▶ 确定性重建先于候选定位和多模态解释。

TELLER 的证据重建：为什么不能只按时间邻近配对？



- ▶ NVTX 提供 engine/framework 语义范围；
- ▶ CUPTI 提供 runtime、driver、GPU kernel 和 correlation ID；
- ▶ stdout/stderr 作为同一执行的文本证据；
- ▶ parent、thread、stream 和 device 关系把跨层事件放回请求树。

TELLER 的算法输出：候选、证据和解释分开



- ▶ 候选排序可以使用层级、时间、资源和日志异常；
- ▶ 证据片段保留原始事件、时间范围和结构关系；
- ▶ 解释层不应创造不存在的 kernel、通信或变更证据；
- ▶ 平台把这些结果挂到 'incident_id' 和请求上下文中。

AlertGuardian：告警系统本身也需要完整生命周期治理



- ▶ 在线图模型先处理高吞吐告警噪声；
- ▶ RAG/LLM 只处理剩余关键告警的上下文摘要；
- ▶ 离线多智能体更新去重、聚合和规则条件；
- ▶ 复核智能体和 SRE 审批保护关键告警不被错误压制。

ChangeRCA: 把软件变化作为独立诊断证据

ChangeRCA: Finding Root Causes from Software Changes in Large Online Systems

2-9

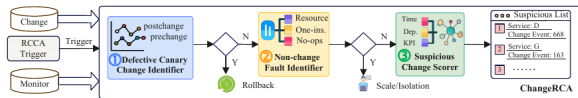


Fig. 6 ChangeRCA framework

1. 先在变更空间中筛选候选；
2. 用 canary、对照组或 DiD 比较变化前后；
3. 融合代码、配置、部署、指标和日志；
4. 输出候选变化、影响范围和置信度；
5. 不把“时间上最近的发布”直接当成根因。

从论文到平台：五种证据如何组合？

论文/方法	主要对象	核心证据	平台接口
Minder	训练机器	同伴、持续性、指标优先序	machine_group、metric_window、fault_candidate
Nezha ChangeRCA	微服务事件 软件变化	指标、日志、Trace、事件图 canary、DiD、变更前后对照	event_graph、pattern、rank change_id、control_group、impact
AlertGuardian	告警生命周期	共现图、噪声节点、知识与反馈	alert_cluster、rule_version、review
TELLER	LLM 请求	跨层调用链、kernel、通信和日志	request_tree、span、evidence

综合判断 论文贡献不应被压缩成“一个分数”；平台要保留它们的对象粒度、证据类型和失败边界。

算法与论文的统一接口：Evidence Bundle

证据包（简化模型）

incident_id、object_scope、time_window、method、candidate[]、evidence[]、confidence、missing[]、next_check。

- ▶ ‘method’ 说明是 Minder、Nezha、ChangeRCA、TELLER 还是基线规则；
- ▶ ‘candidate[]’ 只表示候选，不把候选提升为事实；
- ▶ ‘missing[]’ 显式记录缺失指标、日志、Trace 或变更；
- ▶ ‘next_check’ 提供低风险、可回滚的下一步验证。

这是把论文算法接入平台的关键：统一输出格式，保留方法差异和证据来源。

论文局限：系统论文的数字不能直接迁移到企业

数据集 故障类型、标签质量和采样率不同。

拓扑 论文环境的服务/机器关系更稳定。

开销 采集、索引、模型和查询成本未必相同。

决策 检测指标改善不等于 MTTR 或安全性改善。

1. 先复现输入、窗口、标签、对照和评价定义；
2. 再在本企业数据上做 shadow mode；
3. 通过回放和人工复核观察误报、漏报和证据缺口；
4. 最后才决定是否进入建议或受控执行模式。

研究拓展小结：哪些证据进入哪个平台接口？

1. 先把证据拿全：Minder 的同伴、TELLER 的跨层调用链都说明上下文决定可诊断性；
2. 再把候选排序：Nezha、ChangeRCA 和告警治理方法把时间、拓扑、变更与反馈组合起来；
3. 最后才谈自动化：AlertGuardian 和操作前检查强调高吞吐处理与人工审批的分层。

算法与适用范围 算法定义计算过程；系统实验检验适用场景、效果与限制。

故障窗口推演：数据库写成功，但事件没有发出去

课程设计：事件状态与待发送记录写在同一个本地事务中，由独立发送器投递消息。

故障位置	持久状态	恢复动作	仍需处理的问题
事务提交前崩溃	两条记录均不生效	重新提交同一事件	请求键需稳定
提交后、发送前崩溃	状态与待发送记录都在	发送器重试投递	消息会晚到
发送后、确认前崩溃	消息可能已被消费	可能再次投递	消费者必须去重
消费后外部动作	本地无法撤销外部副作用	查询动作账本	执行端幂等与验证

消费者可将“已处理 event ID”和本地状态更新放进同一事务。

推导与判断 该设计解决本地状态与待发消息的原子记录问题，并不自动提供端到端恰好一次；外部写动作仍需独立幂等协议。

算例使用假设数据。

下一讲预告：大模型驱动的平台架构进阶

模型入口 自然语言、结构化意图和工具选择。

知识入口 RAG、记忆、案例和版本化上下文。

治理入口 Agent 运行时、评测、成本、策略和人工接管。

本讲明确对象、事件、流程和权限；下一讲讨论大模型怎样参与平台决策。

参考材料与论文

1. 《中山大学研究生课程教学大纲-课程表修订版-20260704》;
2. 课程案例中的平台架构、隔离、工作流和安全执行设计;
3. ‘课件/tooling.pdf’、‘课件/agentworkflows.pdf’、‘课件/Overview.pdf’、‘课件/ai-agents.pdf’;
4. ‘课件/ai-infra-engineer-learning-main.zip’ 的 Monitoring & Observability、ML Monitoring 和平台工程模块;
5. 第 3、4、9、10 讲课件及对应材料分析文档;
6. 蓝鲸开源平台社区文档: 用于产品模块和工程实践的进一步核对;
7. Deng et al., *Minder*, NSDI 2025; Xu et al., *TELLER*, ASE 2026 (已录用);
8. Yu et al., *AlertGuardian*, ASE 2025; Yu et al., *ChangeRCA*, FSE 2024;
9. Chen et al., *Nezha*, ESEC/FSE 2023; Yu et al., *MicroRank*, WWW 2021;
10. Tuli et al., *TranAD*, PVLDB 2022; Xu et al., *Anomaly Transformer*, ICLR 2022;

课程案例和论文数字均保留适用条件; 不要将论文实验或案例界面外推为通用生产保证。

进一步分析

1. 如果只能先做一个模块，你会先做 CMDB、事件中心还是作业平台？为什么？
2. 如果 Trace 数据缺失 30%，平台应该怎样表达不确定性？
3. 什么动作永远不应该由平台自动执行？如何验证这个边界？
4. 你设计的平台如何避免三个月后重新退化为群聊和个人脚本？

开放问题 平台工程的难点不是“把组件连上”，而是让数据、责任和结果在变化中仍然对齐。

谢谢

问题、平台蓝图与工程实践讨论